Generating Data

```
import warnings
warnings.filterwarnings('ignore')

!pip install pyspark
from pyspark.sql import SparkSession
from pyspark.sql.functions import expr # Import expr for use in withColumn

spark = SparkSession.builder \
    .appName("BDA_Experiment") \
    .config("spark.jars.packages", "org.apache.spark:spark-avro_2.12:3.5.0") \
    .getOrCreate()

df_large = spark.range(0, 80000000)

df_large = df_large.withColumn("TransactionID", expr("concat('T', id)")) \
    .withColumn("ProductID", expr("concat('P', cast(rand()*100 as int))")) \
    .withColumn("City", expr("""
        case
            when rand() < 0.4 then 'Hyderabad'
            when rand() < 0.7 then 'Mumbai'
            when rand() < 0.85 then 'Delhi'
            else 'Vizag'
        end
    """)) \
    .withColumn("Quantity", expr("cast(rand()*5 + 1 as int)")) \
    .withColumn("Price", expr("cast(rand()*1000 + 100 as int)"))

df_large = df_large.drop("id")

df_large.write.mode("overwrite").csv("sales_csv")
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
```

Converting to File Formats

CSV

```
df_large.write.mode("overwrite").csv("sales_csv")
```

```
!du -sh sales_csv
```

```
2.1G    sales_csv
```

JSON

```
df_large.write.mode("overwrite").json("sales_json")
```

```
!du -sh sales_json
```

```
6.8G    sales_json
```

```
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
```

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("BDA_Experiment") \
    .config("spark.jars.packages", "org.apache.spark:spark-avro_2.12:3.5.0") \
    .getOrCreate()
```

AVRO

```
df_large.write.format("avro").mode("overwrite").save("sales_avro")
```

```
!du -sh sales_avro
```

```
966M    sales_avro
```

PARQUET

```
df_large.write.mode("overwrite").parquet("sales_parquet")
```

```
!du -sh sales_parquet
```

```
588M    sales_parquet
```

Define Benchmark Function

```
import time
from pyspark.sql.functions import col

def benchmark(path, format_type):
    start = time.time()

    if format_type == "csv":
        df = spark.read.option("header", "false").csv(path)
        df = df.toDF("TransactionID", "ProductID", "City", "Quantity", "Price")
        df = df.withColumn("Price", col("Price").cast("double"))
    else:
        df = spark.read.format(format_type).load(path)

    df.groupBy("City").sum("Price").collect()

    end = time.time()

    return round(end - start, 2)
```

Run Benchmark for Each Format

```
csv_time = benchmark("sales_csv", "csv")
json_time = benchmark("sales_json", "json")
avro_time = benchmark("sales_avro", "avro")
parquet_time = benchmark("sales_parquet", "parquet")

print("CSV Time:", csv_time)
print("JSON Time:", json_time)
print("Avro Time:", avro_time)
print("Parquet Time:", parquet_time)
```

```
CSV Time: 111.24
JSON Time: 281.4
Avro Time: 69.25
Parquet Time: 9.42
```

```
df_large.groupBy("City").sum("Price")
```

```
DataFrame[City: string, sum(Price): bigint]
```

File Size Comparison

```
!du -sh sales_csv
!du -sh sales_json
!du -sh sales_avro
!du -sh sales_parquet
```

```
2.1G    sales_csv
6.8G    sales_json
966M    sales_avro
588M    sales_parquet
```

```
df_large.groupBy("City").count().show()
```

```
+---------+--------+
|     City|   count|
+---------+--------+
|    Vizag| 2162387|
|   Mumbai|33599228|
|    Delhi|12239063|
|Hyderabad|31999322|
+---------+--------+
```

Implement Skew Fix (Salting) Practically

Add Salt Column

```
from pyspark.sql.functions import floor, rand, concat_ws

# Add salt column (0 to 3)
df_salted = df_large.withColumn("salt", floor(rand() * 4))

# Create new key: City + salt
df_salted = df_salted.withColumn("City_salted", concat_ws("_", "City", "salt"))

df_salted.show(5)
```

```
+-------------+---------+---------+--------+-----+----+-----------+
|TransactionID|ProductID|     City|Quantity|Price|salt|City_salted|
+-------------+---------+---------+--------+-----+----+-----------+
|           T0|       P9|   Mumbai|       5|  757|   2|   Mumbai_2|
|           T1|      P45|Hyderabad|       4|  615|   3|Hyderabad_3|
|           T2|      P45|Hyderabad|       5|  119|   2|Hyderabad_2|
|           T3|      P15|   Mumbai|       5|  253|   3|   Mumbai_3|
|           T4|       P3|   Mumbai|       3| 1045|   0|   Mumbai_0|
+-------------+---------+---------+--------+-----+----+-----------+
only showing top 5 rows
```

First Level Aggregation (Split Load)

```
partial_agg = df_salted.groupBy("City_salted").sum("Price")
```

Remove Salt (Final Aggregation)

```
from pyspark.sql.functions import split

final_agg = partial_agg.withColumn("City", split("City_salted", "_")[0]) \
    .groupBy("City") \
    .sum("sum(Price)")

final_agg.show()
```

```
+---------+---------------+
|     City|sum(sum(Price))|
+---------+---------------+
|    Vizag|     1296429046|
|   Mumbai|    20143051362|
|    Delhi|     7337229810|
|Hyderabad|    19185283542|
+---------+---------------+
```

Measure Time Difference

Benchmark normal groupBy:

```
import time

start = time.time()
df_large.groupBy("City").sum("Price").collect()
print("Normal Time:", round(time.time() - start, 2))
```

```
Normal Time: 6.19
```

Benchmark salted version:

```
start = time.time()
final_agg.collect()
print("Salted Time:", round(time.time() - start, 2))
```

```
Salted Time: 20.95
```

Adaptive Format Selection Framework

Workload A — Low Column Usage (2 columns)

```
def workload_A(df):
    return df.groupBy("City").sum("Price").collect()
```

Workload B — Medium Column Usage (4 columns)

```
from pyspark.sql.functions import sum, count

def workload_B(df):
    return df.groupBy("City").agg(
        sum("Price"),
        sum("Quantity"),
        count("ProductID")
    ).collect()
```

Workload C — Full Row Scan

```
def workload_C(df):
    return df.count()
```

Benchmark Function

```
import time
from pyspark.sql.functions import col

def run_benchmark(path, format_type, workload):

    if format_type == "csv":
        df = spark.read.option("header", "false").csv(path)
        df = df.toDF("TransactionID", "ProductID", "City", "Quantity", "Price")
        df = df.withColumn("Price", col("Price").cast("double"))
        df = df.withColumn("Quantity", col("Quantity").cast("double"))
    else:
        df = spark.read.format(format_type).load(path)

    start = time.time()
    workload(df)
    end = time.time()

    return round(end - start, 2)
```

```
results = {}

formats = {
    "CSV": ("sales_csv", "csv"),
    "JSON": ("sales_json", "json"),
    "Avro": ("sales_avro", "avro"),
    "Parquet": ("sales_parquet", "parquet")
}

for name, (path, fmt) in formats.items():
    results[name] = {
        "Workload_A": run_benchmark(path, fmt, workload_A),
        "Workload_B": run_benchmark(path, fmt, workload_B),
        "Workload_C": run_benchmark(path, fmt, workload_C)
    }
```

```
results
```

```
{'CSV': {'Workload_A': 111.92, 'Workload_B': 134.52, 'Workload_C': 28.38},
 'JSON': {'Workload_A': 161.2, 'Workload_B': 188.46, 'Workload_C': 114.27},
 'Avro': {'Workload_A': 66.99, 'Workload_B': 72.02, 'Workload_C': 45.31},
 'Parquet': {'Workload_A': 10.36, 'Workload_B': 16.57, 'Workload_C': 0.34}}
```

Effective Throughput (GB/sec)

Throughput = File Size / Execution Time

```
sizes = {
    "CSV": 2.1,
    "JSON": 6.8,
    "Avro": 0.966,
    "Parquet": 0.588
}

throughput = {}

for fmt in results:
    full_scan_time = results[fmt]["Workload_C"]
    throughput[fmt] = round(sizes[fmt] / full_scan_time, 3)

throughput
```

```
{'CSV': 0.074, 'JSON': 0.06, 'Avro': 0.021, 'Parquet': 1.729}
```

Estimate Costs

```
def estimate_cost(format_name):

    size = sizes[format_name]   # GB
    time = results[format_name]["Workload_C"]

    io_cost = size
    cpu_cost = time - size      # rough approximation
    shuffle_cost = results[format_name]["Workload_A"]

    return {
        "IO": io_cost,
        "CPU": cpu_cost,
        "Shuffle": shuffle_cost
    }
```

Compute Total Cost

```
def total_cost(costs, alpha=1, beta=1, gamma=1):
    return alpha*costs["IO"] + beta*costs["CPU"] + gamma*costs["Shuffle"]
```

Select Best Format

```
best_format = None
min_cost = float("inf")

for fmt in results:
    c = estimate_cost(fmt)
    t = total_cost(c)

    if t < min_cost:
        min_cost = t
        best_format = fmt

best_format
```

```
'Parquet'
```